Towards the Construction of Global IPv6 Hitlist and Efficient Probing of IPv6 Address Space

Guanglei Song*‡, Lin He*‡, Zhiliang Wang*‡, Jiahai Yang*‡, Tao Jin†‡, Jieling Liu*‡, Guo Li*‡
*Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China
†Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China
‡Beijing National Research Center for Information Science and Technology (BNRist)

***Abstract*—Fast IPv4 scanning has made sufficient progress in network measurement and security research. However, it is infeasible to perform brute-force scanning of the IPv6 address space. We can find active IPv6 addresses through scanning candidate addresses generated by the state-of-the-art algorithms, whose probing efficiency of active IPv6 addresses, however, is still very low.**

In this paper, we aim to improve the probing efficiency of IPv6 addresses in two ways. Firstly, we perform a longitudinal active measurement study over four months, building a high-quality dataset called hitlist with more than 1.3 billion IPv6 addresses distributed in 45.2k BGP prefixes. Different from previous work, we probe the announced BGP prefixes using a pattern-based algorithm, which makes our dataset overcome the problems of uneven address distribution and low active rate. Secondly, we propose an efficient address generation algorithm DET, which builds a density space tree to learn high-density address regions of the seed addresses in linear time and improves the probing efficiency of active addresses. On the public hitlist and our hitlist, we compare our algorithm DET against state-of-the-art algorithms and find that DET increases the de-aliased active address ratio by 10%, and active address (including aliased addresses) ratio by 14%, by scanning 50 million addresses.

***Index Terms*—IPv6, Hitlist, Address Probing, Network Measurement**

I. Introduction

Network measurement and research has made sufficient progress in IPv4, benefiting from fast scanning entire IPv4 Internet in just a few minutes [1] [2]. Researchers can fast obtain network data to support various applications, including network service measurement [3], network topology discovery [4], network security analysis [5] [6], address analysis [7], host fingerprint [8], and vulnerability analysis [9].

With the growing address exhaustion of IPv4 [10], IPv6 is being deployed increasingly commonly around the world, and this trend will accelerate. More than 25% of Google's users access its services via IPv6 in December 2019 [11]. As of December 2019, more than 70% of ASes have deployed IPv6, and about 24.8% of Alexa Top 1000 websites currently are reachable over IPv6. Global IPv6 traffic has grown more than 5000% since World IPv6 Launch began on June 6, 2012 [12].

Because of the deployment of IPv6, brute-force scanning of all IPv6 addresses is infeasible. For example, using 10 Gigabit link and ZMapv6 [8], it will take at least hundreds of millions of years to scan all IPv6 addresses. To overcome this challenge, researchers characterize and model the structure of seed address space, and generate candidate addresses that may have a higher probability to be active, so as to reduce the probing scope.

Usually, the seed addresses are obtained from target lists of IPv6 addresses called hilist. However, the distribution of the existing public hitlists in the BGP prefix space is uneven. For example, 80% of the addresses in the public hitlist [8] are only distributed in 1% of the BGP prefixes. Besides, in the hitlist, the number of active addresses per day is less than 7M, and the active address rate is low. Low-quality hitlist reduces the efficiency of IPv6 address probing.

There are mainly two kinds of methods for probing active IPv6 addresses: address internal structure analysis [7] and seed address fingerprint discovery by clustering analysis [8] [13] [14]. The former refers mainly paid attention to finding the internal structural features of the allocated addresses in the network space and used analyzed features to generate addresses. Information entropy [7] was used to segment the addresses by entropy value, and disscover the relationship between different fragments. However, because there are few seed addresses, it is impossible to study the internal structural features of the addresses effectively. Therefore, address discovery based on the address structure is inefficient. The latter focuses on how to improve the clustering algorithm for finding more active addresses. Seed addresses were clustered to generate different clusters, the possible active addresses were generated based on the clusters. However, this method has two limitations. First, seed addresses are distributed in a few BGP prefixes, so active addresses can only be found in a small number of prefixes. Second, there are some flaws in the representative algorithms include 6Gen [13] and 6Tree [14]. 6Gen cannot be used for large-scale address discovery due to excessive time complexity. 6Tree only considers the hierarchical structure of addresses and limits the scope of address scanning under the same budget. Therefore, the method limits the efficiency of address probing.

To address the above issue, we collect a hitlist with larger quantity and higher quality and design a more efficient address generation algorithm. We first propose a pattern-based address discovery algorithm called Random-Bytes. We perform Random-Bytes in each BGP prefix to collect IPv6 addresses from overcoming the uneven distribution of IPv6 hitlist. At

Corresponding author: Jiahai Yang (yang@cernet.edu.cn).

978-1-7281-6887-6/20/$31.00 © 2020 IEEE

the same time, we collect IPv6 addresses from multiple data sources to enrich our hitlist. Then, we propose DET, which identifies high-density address regions in seed addresses in linear time and dynamically generates addresses. We assume that active IPv6 addresses are more likely to exist in high-density regions of seed addresses, and construct an IPv6 density space tree to store the density distribution characteristics of seed addresses. The root node of the density space tree represents the entire seed addresses, and each leaf node represents a high-density region. To avoid splitting high-density address regions as much as possible, we creatively propose that node is split at the position with the smallest entropy value, and provide theoretical proof. DET uses top-down divisive hierarchical clustering (DHC) [14] to learn the active IPv6 addresses space distribution structure and build a density space tree based on the vectors of seed addresses. Then we use dynamic scanning tools [14] to generate addresses in the density space tree. As a result, DET dramatically improves the efficiency of IPv6 address scanning.

Contribution. We make the following contributions:

- **Novel address collection method**: We propose a Random-Bytes algorithm to generate target addresses in each BGP prefix and collect active IPv6 addresses. Finally, we collect 870M active IPv6 addresses.
- **Greater quantity and higher quality of IPv6 hitlist**: We improve the quantity and quality of IPv6 hitlist. In terms of quality, we have collected a more even distribution of addresses and a higher proportion of active addresses. In terms of quantity, we have collected a larger IPv6 hitlist that has 1.3 billion addresses.
- **More efficient address generation algorithm**: We propose a more efficient address generation algorithm called DET, which learns the address distribution characteristics in linear time and increases the hit rate of active addresses by about 10% compared with state of the art. Besides, we share our code and hitlist at:
  https://github.com/sixiangdeweicao/DET

The remainder of this paper is organized as follows. In Section II, we introduce the background of active IPv6 address probing. Related work is reviewed in Section III. Section IV presents a framework of active address probing. In Section V, we describe how to build a hitlist with greater quantity and higher quality. In Section VI, we present the details of DET algorithm. We evaluate hitlist quality, DHC time performance, and target generation effect in Section VII. In Section VIII, we present ethical considerations for address scanning. Future work is discussed in Section IX. Finally, we conclude this paper in Section X.

II. Background

In this section, we explain the related terms mentioned and discuss IPv6 address characteristics. In Table I, we describe the terminology used throughout the paper.

TABLE I
EXPLANATION OF TERMINOLOGY

Terminology	Explanation
hitlist	IPv6 address list extracted from multiple data sources
seed address	Active address as input of address generation algorithms
target address	Possible active address generated by a address generation algorithm
budget	The probing number of target addresses
nd_stable	Address is continuously active for n days
aliased prefix	A prefix which is assigned to one active host
aliased address	An address in an aliased prefix

The new protocol makes IPv6 address have the following characteristics. (1) The hugeness of IPv6 address space: The IPv6 address space is 2^{128}, which is 2^{96} times the IPv4 address space. The huge address space solves the problem of exhausting IPv4 address resources; (2) Diversity of IPv6 address IID allocation methods: Common IID allocation types include random [15], low-byte [15], IPv4 embedding [16], Teredo tunnel [17], EUI-64 embedding [18], Modified EUI-64 [18], Wordy [15], Ports [19], etc; (3) Sparsity of IPv6 address space. The currently active IPv6 addresses are only a small part of the IPv6 space. Assuming IPv6 users are equivalent to IPv4, the usage rate of the IPv6 address space is only $\frac{1}{2^{96}}$. Such features of IPv6 address lead to stronger concealment in IPv6 space. It makes finding active hosts become more complicated, which improves the security to a certain level; however, it increases the difficulty of network measurement research.

III. Related Work

In this section, we introduce previous work on IPv6 address scanning. Related work falls in two areas: (1) extracting IPv6 addresses from multiple sources to construct hitlist, (2) designing algorithms to generate target addresses.

A. Extracting IPv6 Addresses

Querying DNS AAAA records to obtain IPv6 addresses is an effective method to obtain IPv6 addresses [20] [21] [22] [23] [8]. Strowes et al. [20] performed brute-force reverse DNS queries on the entire IPv4 address space, and then queried AAAA records on the results to obtain 96.5k globally routable IPv6 addresses. Fiebig et al. [21] proposed an innovative approach that used DNS's existence semantics (NXDOMAIN) to collect 5.8M IPv6 addresses without restricted network vantage points. Borgore et al. [22] traversed DNSSEC-signed IPv6 reverse zones to obtain 2.2M IPv6 addresses. Gasser et al. [8] published a hitlist that collected 58.5M IPv6 addresses from public data sources and tracerouted the collected addresses to obtain router addresses. However, the address distribution of their hitlist is uneven. 80% of the addresses in their hitlist are only distributed in 1% of the BGP prefixes. Compared with their work, we probe active addresses in the announced BGP prefixes using a pattern-based method, which overcomes the problem of uneven addresses distribution and improves the probing efficiency.

B. Generating Target Address

Generating target addresses to reduce the scanning space is an effective method for IPv6 active address probing. Generally, there are mainly two ways to generate target addresses.

Internet address internal structure: The first one is to generate target addresses by discovering internal structure based on the analysis of IPv6 subsets known to be active [7], [8]. Foremski et al. [7] proposed Entropy/IP, which employed a combination of information-theoretic and Bayesian Network to model IPv6 addresses probabilistically. Gasser et al. [8] extended the scope of prefix space for Entropy/IP and improved the efficiency of address generation. Entropy/IP studied the structure information of the allocated addresses in the network. However, if seed addresses are not enough or unevenly distributed in subnets, it can not work effectively. DET borrows the idea from Entropy/IP and uses information entropy to construct the density space tree.

Seed address fingerprint discovery: The second one is to generate target addresses by discovering fingerprints of seed addresses [24] [13] [14]. Ullrich et al. [24] proposed a pattern-based probing technology that greedily recursively added a bit to the pattern to include more seed addresses. It found 50%-100% more addresses than brute force generation based on address configuration. However, the process of pattern learning requires manual intervention and is influenced by human factors. DET can probe active addresses automatically without human intervention. Murdock et al. [13] used density fingerprint of seed addresses and proposed 6Gen to discover high-density regions of seed addresses. It assumed that more active addresses could be found in high-density regions and used a hamming distance to measure the distance between seeds. 6Gen maintained the maximum density region and the minimum scale to generate high-density address regions using agglomerative hierarchical clustering (AHC). 6Gen outperformed Entropy/IP by 1-8 times. However, the time complexity of 6Gen is very high, and it does not scale to large-scale address generation. DET builds a density space tree to discover high-density regions in linear time. Liu et al. [14] proposed 6Tree that performed DHC on vectors of seed addresses to construct a space tree and used a dynamic address scanning mechanism to improve the efficiency of discovering active addresses. However, the hierarchical space tree of addresses limits the scope of address probing. Besides, 6Tree cannot work in the prefixes without seed addresses. Different from 6Tree, DET combines density and hierarchy fingerprints to build a dynamic space tree, which is free from the limitation of determined probing scope under the same budget.

IV. Framework of Active Address Probing

In this section, we propose a framework for IPv6 active address probing, which systematically performs address probing on IPv6 address space.

As shown in Fig. 1, IPv6 active address probing includes four modules: construction IPv6 hitlist, target addresses generation, addresses scanning, and aliased prefix detection. When probing active IPv6 addresses, the IPv6 hitlist module is first used to construct a hitlist from multiple data sources. The target address generation module uses the seed addresses provided by the hitlist to generate the target addresses. Then, the address scanning module uses a high-speed address scanning tool (e.g., ZMap) to scan target addresses and discover active IPv6 addresses. Finally, the aliased detection module is used to remove aliased prefixes and addresses. Besides, newly discovered active addresses are added to the hitlist to enrich the hitlist further. We perform active address probing following the probing framework above. In this paper, we mainly focus on building a more valuable hitlist (Section V) and designing a faster and more efficient address generation algorithm (Section VI).

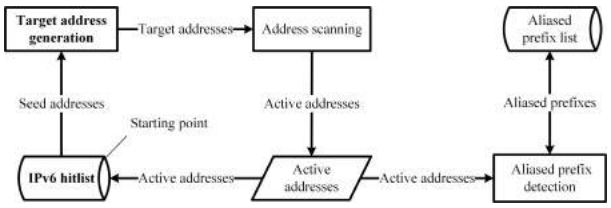


Fig. 1. Framework of IPv6 active address probing

V. IPv6 Hitlist

As mentioned in Section III-A, although Gasser et al. [8] published a hitlist, it is uneven. Hence, their hitlist can not be representative of the internet as a whole. Furthermore, it is not feasible to generate target addresses using address generation algorithms if there don't exist seed addresses in BGP prefixes.

In this section, we describe a collection method of how to construct a global IPv6 hitlist with greater quantity and higher quality. First, we introduce the Random-Bytes algorithm for discovering active IPv6 addresses based on announced BGP prefixes. Then, we obtain IPv6 addresses from multiple data sources. Finally, we continuously probe the balanced IPv6 hitlist to maintain addresses fingerprints about protocol response capability and time active characteristic.

A. Random-Bytes

BGP prefixes represent the currently announced and globally routing addresses, which is the small subset of the entire IPv6 space. To make the hitlist more evenly distributed in the BGP prefixes, we implement an effective method called Random-Bytes to obtain active addresses. It makes full use of the announced BGP prefixes and generates IPv6 addresses that may be active in each BGP prefix.

When analyzing the IPv6 addresses extracted from public sources, we find that there are a large number of addresses, in line with the address allocation policies recommended in RFC7707 [15]. The addresses in Fig. 2(a) can be represented by the pattern 2001:ee0:3200::****. "*" in the pattern means a wildcard, which represents any value from 0 to f. This structure is a typical low-byte address allocation method. The addresses in Fig. 2(b) and Fig. 2(c) can be represented by the pattern 2001:ac8:23:****::1 and 2001:ac8:23:****::0, respectively.

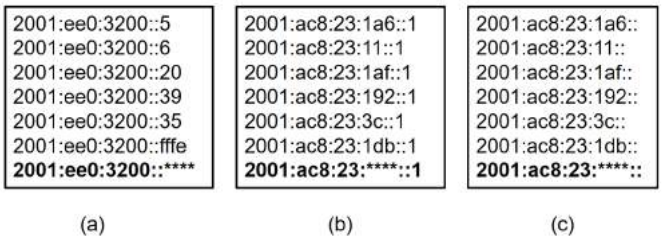


Fig. 2. The patterns of random-bytes in IPv6 addresses

We refer to these patterns as random-bytes. We define the 16 bits between two colons as double-bytes, so IPv6 addresses can be represented by 8 double-bytes. As the algorithm goes, the variable double-bytes position is moved to the right in order, in double-bytes steps. The addresses range 0000-ffff are generated invariable double-bytes which begin the first double-bytes space closest to the prefix, the last double-bytes takes 0 or 1, and the other position's value is 0. For a BGP prefix with a length of p, there are $[2 \times (\left\lceil \frac{128-p}{16} \right\rceil) + 1] \times 2^{16}$ IPv6 addresses generated in the BGP space.

B. Building Hitlist

To collect hitlist that is more evenly distributed and contains more announced BGP prefixes with seed addresses, we collect IPv6 addresses from multiple sources, for which we provide an overview in Table II and the quantity and quality of the hitlist are evaluated in Section VII-A. The sources we leverage are as follows:

Public: Although Gasser et al. [8] disclosed the extraction of IPv6 addresses from public data sources, they have limited downloads and can only get part of the data. Therefore, we reproduced the method of Gasser and extracted IPv6 addresses based on the data sources including domain lists [25] [26] [27], Rapid7 [28], TLDR project [29], Bitnodes API [30], IPmap project [31], TLS certificate logs, and then tracerouted the collected addresses to obtain router addresses. We probed daily from December 4th, 2019 to February 10th, 2020, and collected 61M IPv6 addresses, which include servers, routers, and clients.

Active: To obtain IPv6 addresses in each BGP prefix, we used Random-Bytes to generate addresses for each announced BGP prefix. We used ZMapv6 to perform ICMPv6 and TCP 80 port scanning on the target addresses. From October 1st, 2019 to January 31st, 2020, we obtained 870 million unique IPv6 addresses.

Passive: China Education and Research Network (CERNET2) is the world's largest IPv6-Only educational network. It is supported by China Next Generation Internet (CNGI) [32]. We mirrored NetFlow traffic from the CERNET2 network exit point and parsed the NetFlow packets to extract IPv6 addresses. From August 1st, 2019 to December 31st, 2019, we collected 340 million unique IPv6 addresses.

TraceIP: Finally, we used scamper [33] to perform traceroute measurement on the IPv6 addresses obtained from active and passive probing to extract router addresses. From December 1st, 2019 to February 15th, 2020, we obtained 49M unique IPv6 addresses. The number of router addresses adequately reflects the progress of IPv6 deployment.

TABLE II
OVERVIEW OF OUR IPV6 HITLIST ON FEBRUARY 16, 2020

Name	Nature	IPs	#ASes	#PFXes	#$\overline{\text{PFXes}}$[1]
Public	Mixed	61M	12.4k	26.3K	25.1k
Active	Mixed	870M	14.1k	35.3k	34.1k
Passive	Mixed	340M	4.2k	8.3k	7.6k
TraceIP	Routers	49M	8.5k	12.6k	11.7k
Total	**Mixed**	**1.3B**	**16.5k**	**45.2k**	**44.1k**

[1] Removing aliased prefixes using aliased prefix detection

TABLE III
NOTATIONS

Notation	Definition
ω	Address vectors corresponding to each node in density space tree
β	IPv6 granularity or base bits of every dimension
b	The limitation of IPv6 space probe number
δ	Maximum number of seed addresses contained in leaf nodes
d	The dimensions of IPv6 address vectors
θ	Active IPv6 seed addresses
Δ	Seed address vectors

C. Address Fingerprints

Due to the lack of address fingerprints, current efficient machine learning and deep learning technologies cannot be applied to address prediction. So we build a fingerprint database of IPv6 addresses. We maintain the time and protocol fingerprints of the IPv6 hitlist. The time fingerprint refers to whether every IPv6 address in the hitlist is active every day. Therefore, we are probing every IPv6 address in the IPv6 hitlist daily to detect whether it is active from January 19th, 2019. The protocol fingerprint refers to the response to different protocols of each IPv6 address. We send ICMPv6, TCP/80 (HTTP, HTTPs) and UDP/53 (DNS) probes to maintain the response of addresses to different protocols.

VI. DET ALGORITHM

In this section, we introduce our DET algorithm. We first present an overview of the algorithm. Then, we propose a novel space tree splitting indicator based on the smallest entropy and introduce the details of the density space tree generation. Finally, we discuss the characteristics of the space tree and optimization based on implementing to operate it on real-world datasets. The relevant notations used throughout the DET description are defined in Table III.

A. Overview

In Section III-B, we discuss the shortcomings of current algorithms. To overcome these problems, we propose an

algorithm that builds a density space tree to find high-density regions of seed addresses in linear time and dynamically generates target addresses. We assume that high-density regions of seeds are associated with high-density regions of active addresses. The root node of the space tree represents the entire seed addresses, and the leaf nodes represent high-density regions of the seed addresses. We astutely find and split the node in the dimension with the smallest entropy value to ensure that high-density regions are at the leaf nodes or the same branch of the space tree.

DET first uses divisive hierarchical clustering (DHC) to build a density space tree from top to bottom. The algorithm operates iteratively. First, DET converts seed addresses to high-dimension vectors and builds the root node by seed address vectors. Then, it splits the node to generate child nodes in the dimension with the smallest entropy and splits the address vectors corresponding to the node into child nodes, until address vectors in the generated child node are less than the threshold (δ). After building the density space tree, DET dynamically generates target addresses in high-density regions of seed addresses by the dynamic generation tool [14].

Our space tree solves the clustering algorithms' problems well because it satisfies the following characteristics:

1) The time complexity of clustering is linear;
2) Avoiding splitting high-density regions, and ensuring that high-density regions are at the leaf nodes or the same branch of the space tree;
3) More realistic fitting of the real IPv6 address space distribution;
4) Lifting budget limitations and scanning larger IPv6 address space.

The motivation behind prioritizing high-density region is budget conservation, as clusters of more "hot spot" regions form smaller regions that consume less budget.

B. Splitting Indicator: The Smallest Entropy

The density space tree's nodes have the high-density characteristic. When splitting nodes, we consider how to split the current node to make the high-density region addresses in the same one child node. Within an acceptable time range, we use a "heuristic" indicator and select the dimension with the smallest entropy value of the current seeds set as the splitting indicator.

Assume that the active address vectors in the current node is ω, including M IPv6 active addresses. Any ω's subset that satisfies high-density characteristics is ω_i, which has m IPv6 active addresses. Without loss of generality, we assume a certain stable dimension x and a variable dimension y in ω_i and consider the entropy values contribution about ω_i to the x and y as $E(x)$, $E(y)$ in ω. ω_i has k values in dimension y, and the corresponding number is m_i (i = 1, ..., k). The relationship between $E(x)$ and $E(y)$ is proved as follows:

$$if \quad p = \frac{m}{M}, \quad p_i = \frac{m_i}{M} \quad then \quad p = \sum_{i=1}^{k} p_i \tag{1}$$

$$E(x) = -p \log_2 p \tag{2}$$

$$E(y) = -\sum_{i=1}^{k} p_i \log_2 p_i \tag{3}$$

$$\begin{aligned} E(x) &= -p \log_2 p \\ &= -\sum_{i=1}^{k} p_i \log_2 p \\ &< -\sum_{i=1}^{k} p_i \log_2 p_i = E(y) \end{aligned} \tag{4}$$

We can derive the quantitative relationship that $E(x) < E(y)$ from Equation (1), (2), and (3).

Because $E(x) < E(y)$ is proved from Equation (4), the entropy contributed by the stable dimension to ω is smaller than the variable dimension of ω_i. Generally speaking, if the entropy value is small in a dimension of the current address vectors, it means that the dimension of current address vectors has more values from the subsets where the dimension is stable (there may be some counter-examples), and has fewer values from the subsets where the dimension is variable. So it can get the smallest density loss to divide the current seed set in this dimension where entropy value is smallest. Therefore, we select the dimension with the smallest entropy value of the current node as the split indicator.

C. Density Space Tree Generation

Algorithm 1 sheds light on how to generate density space tree.

First, we vectorize the IPv6 seed address set (e.g., β = 4), and then perform DHC on the address vectors. We initialize the root node of the space tree, which corresponds to the entire active seed addresses. Then, we perform DHC clustering on the root node to generate child nodes based on the splitting indicator. The seed address vectors corresponding to the node will be split into multiple subsets, each of which is distributed in a child node. Similarly, the new child nodes continue to perform DHC clustering until the number of address vectors corresponding to the child nodes is less than the threshold δ. The node that does not split is called leaf node, which stores high-density region of seed addresses. In the clustering process, when there are multiple equal minimum entropies, the child nodes with the left dimension have a higher priority than the right dimension. Left dimension priority maintains the hierarchical characteristic of the address. With each iteration of node splitting, the child node adds a stable dimension. In the density space tree, the number of a node's stable dimensions is equal to the depth of the node in the space tree. When δ = 1, the depth of the space tree is equal to the dimension of the IPv6 address. We use a stack to store the order in which the dimensions become stable. After a node splits during the clustering process, generating child nodes add the stable

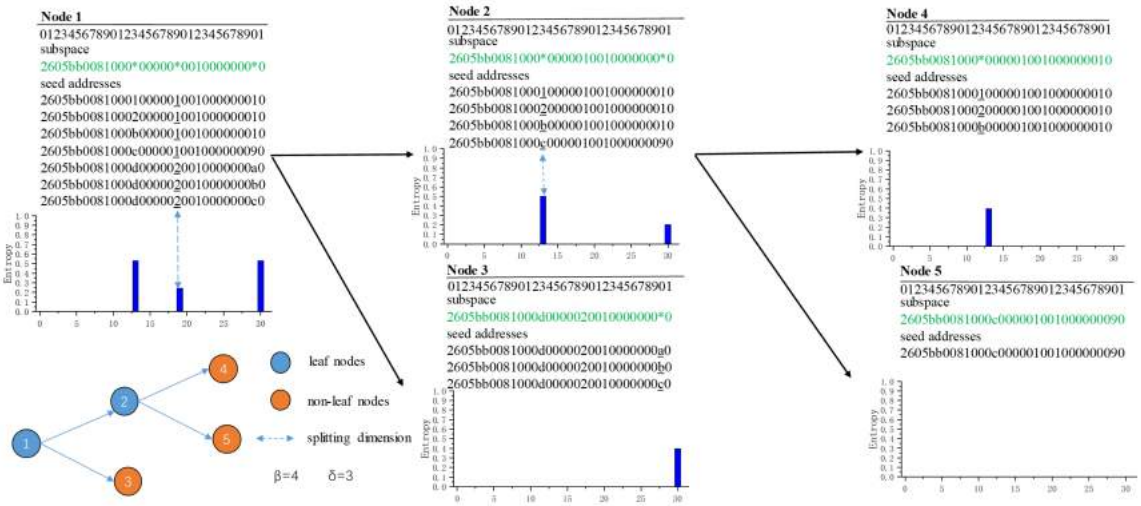


Fig. 3. Density space tree generation process

dimensions to the stack. Note that because of the introduction of the stack, the node with only one child is merged with its child into a node. The density space tree's nodes have the stack attribute with the following benefits: First, stack simplifies the density space tree. Second, fewer nodes reduce the memory consumption caused by the nodes storing corresponding seed vectors. As shown in Fig. 3, we use seven active seed addresses to generate a density space tree, which includes five nodes.

D. Algorithm Characteristics

In this section, we analyze some characteristics of the DET algorithm.

Assuming the number of nodes at the same depth of the space tree is l, and the number of seeds for the corresponding l nodes are n_1, n_2,..., n_l. An arbitrary parent node a having k immediate children represented by node b_i ($i \in [1, 2, ..., k]$), and the number of seeds for the node is n. The relationship between the parent node and its children in the space tree is as follows:

(1) The number of stable dimensions of node b_i is at least one greater than the number of stable dimensions of node a.

(2) The seeds of the node a is equal to the sum of the seeds of its all child nodes b_i.

Linear time complexity: The time complexity of calculating the $\frac{128}{\beta}$-dimensional entropy and dividing the current node is $O(n)$. Considering the worst case, the stable dimensions of the root node after initialization is 0, and the number of stable dimensions of the child node is equal to the number of stable dimensions of the parent node plus one. We can derive the following equation:

$$N = \sum_{i}^{l} n_i, \tag{5}$$

where N is the number of input seed addresses.

Therefore, the time to split all nodes with the number of stable dimensions f into $f + 1$ is $O(N)$. There are $\frac{128}{\beta}$ possibilities for the stable dimension from 0 to $\frac{128}{\beta}$ in total because the $\frac{128}{\beta}$ dimensions all contain only a single IPv6 address and should not be split. Therefore, a maximum of $\frac{128}{\beta}$ layers of splitting are performed, and the time complexity of each splitting is $O(N)$. The total time complexity is also $O(N)$.

Density characteristics: According to the node splitting indicator proposed in Section VI-B, in each iteration, we split nodes based on the dimension with the smallest entropy value. When the entropy value is zero, the node does not split. So the child node has the same seed addresses as the parent node, and the density remains the largest. When the entropy value is not zero, the number of values (k) in the selected dimension is small, because this dimension has the smallest entropy value. Using greedy theory, the average density of the node seed addresses is proportional to $\frac{n}{k}$. We guarantee that the value of k is the smallest when splitting, so each iteration guarantees the largest average density of the nodes. Throughout the iteration process, we avoid splitting the high-density regions and ensure that the high-density regions are at the leaf nodes or the same branch of the space tree.

More realistic fitting addresses space distribution: When using DET to find a possible active IPv6 address, the probed new active IPv6 addresses are added to the seed addresses. Under a specific budget, the expanded seed address will not change the 6Tree 's probing space scope, but the density space tree will have a structural change due to the change in seed address density.

Larger probing space: With the same budget, the structure of the address space tree in 6Tree [14] remains unchanged except hierarchy expansion. The target addresses are generated in the same address space and are scanned repeatedly, which wastes probe resources. When the address prediction algorithm is executed multiple times, the density space tree structure constructed by DET will change due to the expansion of the seed address, so the IPv6 address detection space will also

Algorithm 1 Density space tree generation

Input: Active IPv6 seed addresses, θ; Maximum number of seed addresses contained in leaf nodes,δ; IPv6 granularity or base bits of every dimension, β;
Output: Density space tree's root node, $root$;

```
Δ = AddresToVecs(θ, β)
root=TreeNode(Δ)        //initialize root node
root = DHC(root, δ, β)
return root

function DHC(node, δ, β)
    vecNum = len(node.ω)
    if vecNum ≤ δ then
        return
    end if
    splitPos = GetSplitPos(node, β)
    if splitPos==-1 then
        return
    end if
    dic_key_vecs = SplitVecs(node, splitPos)
    for key ∈ dic_key_ves do
        new_node = TreeNode(dic_key_vecs[key])
        node.childs.append(new_node)
    end for
    for child ∈ node.childs do
        DHC(child, θ, β)
    end for
end function

function GetSplitPos(node, β)
    bestEntropy, splitPos = float("Inf"), −2
    for i ∈ range(128/β) do
        entropy = node.get_entropy(i)
        if entropy == 0 then
            continue
        else
            if bestEntropy > entropy then
                bestEntropy = entropy
                splitPos = i
            end if
        end if
    end for
return splitPos + 1
end function

function SplitVecs(node, splitPos)
    dic_key_vecs = {}
    for ip ∈ node.ω do
        if ip[splitPos − 1] ∈ dic_key_vecs then
            dic_key_vecs[ip[splitPos − 1]].append(ip)
        else   dic_key_vecs[ip[splitPos − 1]] = [ip]
        end if
    end for
end function
```

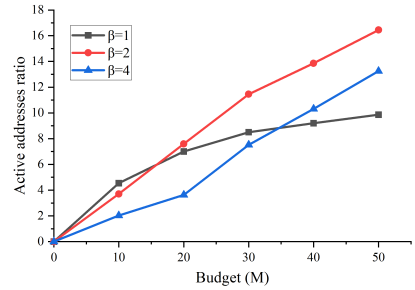


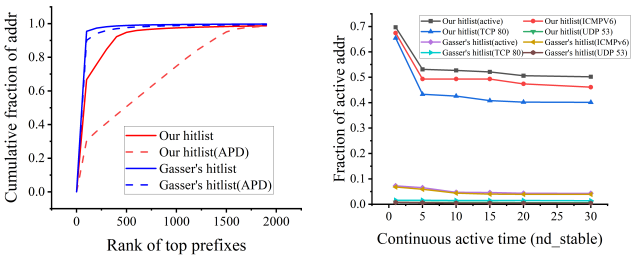
Fig. 4. Prediction results under different granularities. We use the DET algorithm to generate target addresses. Using default 4-bit granularity, the prediction efficiency is not the best, especially when the budget is small (e.g., less than 35M).

change. To a certain extent, the budget limit is lifted, and the larger IPv6 address space is probed.

E. Optimization

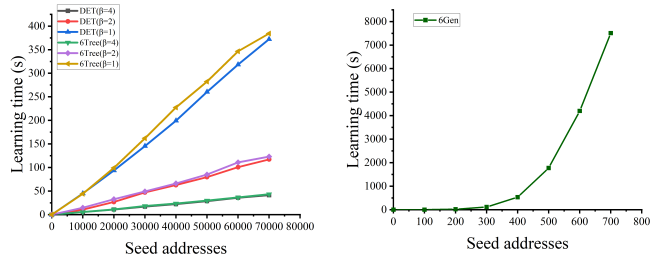
Taking into account the address writing specifications specified by RFC 4291 [18] and the habit of administrators to manually configure addresses, 4 bits are usually used as the granularity of research. So, what impact does the granularity of address research have on target address generation? And what address granularity will produce better results when generating target addresses? In this section, we explore the effect of address granularity and optimize the effect of DET on the real-world dataset.

We use the 2.3M 100d-stable active addresses provided by Gasser as the seed addresses, and execute DET to generate target addresses with different budgets at different address granularities (e.g., β = 1,2,4). Without considering aliased prefixes, the prediction results of the algorithm are shown in Fig. 4. Surprisingly, the algorithm's prediction result is the worst when β = 4 and the budget is less than 35M, which is very different from the general assumption. This is related to how addresses are generated in a small range. When a particular dimension is variable in vectors, the address generation range is (0, 2^{β}-1). The larger the β is, the looser the address generation method is, and the smaller the β is, the tighter the address generation method is. The generation of the target addresses is driven by the seed addresses. When the address budget is small, and the address generation by the tighter way (β = 1) is more efficient to find the active addresses. When the address budget is large, the method of tight address generation method (β = 1) limits the space for address generation. At the same time, too loose address generation (β = 4) will lead to a reduction in data-driven effects. According to the experimental results, we propose the following suggestions about configuring the algorithm parameters. When the budget is small, we choose a tight address generation method with β = 1; when the budget is large, we use the slightly looser address generation method with β = 2.



(a) Prefix distribution of two hitlists (b) Active address stability of two hitlists

Fig. 5. Compares the quality with Gasser's hitlist



(a) DET VS 6Tree (DHC) (b) 6Gen density clustering (AHC)

Fig. 6. Time performance comparison of clustering algorithms to learn seeds addresses characteristics in different address generation algorithms including DET, 6Tree and 6Gen.

VII. Evaluation

A. Hitlist's Quantity and Quality

In this section, we compare the collected hitlist with Gasser's hitlist.

Larger quantity of addresses: Table IV shows that we build a larger hitlist which contains 1.3B IPv6 addresses, while Gasser's hitlist only has 61M IPv6 addresses.

Higher quality: The higher quality of our hitlist is reflected in a more balanced address distribution and a more stable active address set. Fig. 5(a) shows the distribution of active addresses in announced BGP prefixes. A few prefixes contain most of the addresses in Gasser's hitlist. Still, regardless of whether aliases are removed or not, the addresses we collect are more evenly distributed in more BGP prefixes. The seed addresses in the advanced BGP prefix space is a prerequisite for address probing using the address generation algorithm. Therefore, we collect as many seed addresses as possible in each prefix. Table IV shows the distribution of ASes and BGP prefixes. We have collected 18.9k more prefixes and 4.1K more ASs than Gasser's hitlist.

Fig. 5(b) shows the stability of active addresses in both hitlists. The stability and the number of active addresses we collected have greatly improved compared to Gasser's work. We recorded continuously active addresses in hitlist from January 7, 2020, to February 7, 2020. If the probed address responds to any of the ICMPV6, TCP 80, and UDP 53 probing packets, the address is active. We find that more than 50% of addresses in our hitlist have been active for a month, compared with only 3.7% in Gasser 's hitlist. Besides, we find that more than 50% of the hosts where the addresses are located have TCP 80 port open and have been active for a long time. We think these addresses are more likely to be server addresses. A large number of long-term stable addresses make our hitlist more useful.

B. DHC Time Performance

We randomly select different numbers of seed addresses from the data set published by Gasser and compare the time performance of the improved clustering algorithm with the DHC in 6Tree and bottom-up hierarchical clustering based on density in 6Gen. Considering that the clustering algorithm's time complexity of 6Gen is too high, the time complexity of DET and 6Tree is the same. For the convenience of the display, we show the clustering time results of 6Gen separately. In Fig. 6(a), we find that the time performance of DET to construct a density space tree is slightly better than 6Tree because 6Tree requires ordered address vectors, and DET does not. In Fig. 6(a) and Fig. 6(b), we find that DET has dramatically improved the time performance of the clustering compared to 6Gen. From an experimental perspective, we verified that the density hierarchy clustering of DET is linear in time complexity, and it outperforms the state of the art tools.

TABLE IV
Comparison Our hitlist and Gasser's hitlist in address distribution

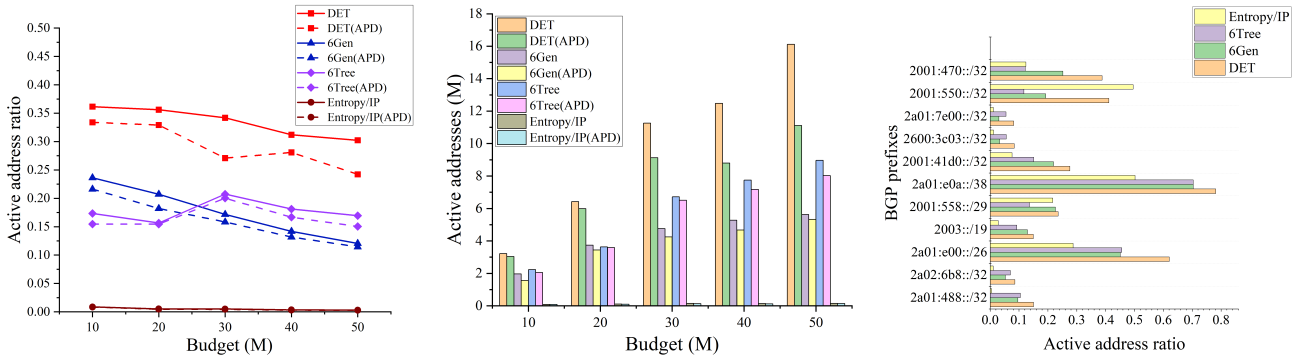
Hitlist	IPs	#ASes	#PFXes	#$\overline{\text{PFXes}}$[1]
Gasser's hitlist	61M	12.4k	26.3k	25.1k
Our hitlist	**1.3B**	**16.5k**	**45.2k**	**44.1k**

[1] Removing aliased prefixes using aliased prefix detection

C. Target Generation Efficiency

We evaluate the efficiency of our algorithm in generating target addresses on the datasets published by Gasser and the seed address set we collected.

We compare DET with 6Tree, 6Gen, Entropy/IP by using 2.3M 10d-stable seed addresses, continuously active from January 10, 2020, to January 20, 2020. Seed addresses are selected from the data set provided by Gasser to generate target addresses under different budgets. We use Multi-Level Aliased Prefix Detection [8] to determine whether the address is aliased or not. In Fig. 7(a), we find that 6Gen and 6Tree are the most advanced algorithms, and Entropy IP is less effective in address generation in previous work. We find that the larger the budget, the smaller the ratio of new active addresses is. As the budget increases, the address generator will generate addresses in suboptimal address regions. Compared with 6Gen and 6Tree, when the budget is from 10M to 50M, DET increases the active proportion of target addresses by more than 10%. Therefore, under the same budget, DET is more efficient to discover active IPv6 addresses and improves the efficiency of the address probing in IPv6.



(a) Target generation in Gasser's hitlist (b) Target generation in our hitlist (c) Target generation in announced prefixes

Fig. 7. Comparison of probing efficiency between DET and other address generation algorithms under different budget. DET finds 10% more active addresses than 6Tree and 6Gen.

To further verify the effectiveness and universality of DET, we randomly select 2.3M 10d-stable IPv6 addresses from our collected seed addresses to generate target addresses. The experimental results are shown in Fig. 7(b), DET generates 50M target addresses, and finds more 7.2M new IPv6 addresses, including aliased addresses and 3.4M new IPv6 addresses excluding aliased addresses than 6Tree.

In Fig. 7(c), we verify DET's address generation efficiency in a single prefix space. We randomly choose BGP prefixes that contain seed addresses in our hitlist. Then we use different addresses generation algorithms to generate the same number of target addresses. We find that the active addresses ratio of DET is higher than 6Gen and 6Tree in the routable prefixes. Compared to Entropy/IP, the generation efficiency of DET has been greatly improved except for 2001:550::/32, which belongs to AS173 and is assigned to Cogent [34] (one of the world's largest Internet Service Providers). Our analysis finds that there are more seed addresses in 2001:550::/32. Therefore, Entropy/IP may have effectively learned the structure of the assigned addresses.

VIII. Ethical Considerations

Before performing global IPv6 address probing, we adhered to ethical conventions for network measurement, including recommendations from Partridge and Allman et al. [35] and Dittrich et al. [36].

First, we consider whether the measurement will cause harm to relevant interest groups, including the impact on active hosts configured with IPv6 and the network where the IPv6 address is located. DET removes the IPv6 addresses that have been probed in the child nodes during the dynamic address detection to avoid repeated probing. Each independent IP only sends three probing packets (ICMPv6, TCP 80, and UDP 53), which slightly affects the network and the host where the IP is located. During active detection, we send probes from CERNET2. We only use one host and limit the packet sending rate to 10M/s, which is unlikely to have a severe network performance impact on the probing network. In passive measurement, we collect NetFlow traffic at vantage points, extract IPv6 addresses from the collected traffic, and split the traffic at vantage points. It will not have a significant impact on network communication status.

At the same time, we have also fully considered privacy and security. When we measure the network, we only measure the performance of the network detection and do not involve any research on related privacy.

IX. Future Work

In this paper, we measure the global IPv6 address space through an improved target address generation algorithm. Here we discuss the directions for further research on IPv6 space measurement.

Further research on address generation algorithms: Although some progress has been made in the study of address generation algorithms, how to efficiently probe the IPv6 address space will be a topic for a long time to come. At the same time, existing address clustering algorithms are all based on statistics, how to apply deep learning and reinforcement learning techniques to study address structure and predict address activity may be considered.

IPv6 alias detection: Although Murdock [13], Gasser [8], and Liu et al. [14] proposed alias detection algorithms, which consider that all active addresses under non-aliased prefixes are extremely unlikely events. However, these methods may cause false positives when the prefix space's addresses are active. Therefore, how to find the aliased prefixes and hosts accurately is difficult in current research. Also, how to propose a more effective fingerprint that uniquely identifies an alias needs further study.

X. Conclusion

In this paper, we construct a IPv6 hitlist with greater quantity and higher quality, which contains 1.3 billion IPv6 addresses and distributes more evenly with more than 50% of active addresses per day. We have collected 18.9k more prefixes and 4.1K more ASs than Gasser's hitlist. We also add active time fingerprint and protocol response fingerprint to hitlist. Considering IPv6 address space cannot be exhaustively

probed like IPv4, we propose DET, which is efficient to generate and probe target addresses. DET uses the smallest entropy value as a splitting indicator to ensure that high-density regions are not split. The density space tree dynamically improves the probing efficiency, better reflects the characteristics of the address density distribution, and can detect a larger address space under the same budget. At the same time, it can learn the address space distribution characteristics in linear time compared with 6Gen's $O(n^3)$ and 6Tree's $O(nlog(n))$ (consider address ordering). In the real IPv6 network, we use a 2.3M seed address to discover 16M active IPv6 addresses (budget = 500M). Besides, we use DET to find new active IPv6 addresses every day, and dynamically and continuously accumulate hitlist data.

Our work on global IPv6 space exploration has important implications for future work. We probe active IPv6 addresses and provide data for measurements such as topology measurement and network mapping to help researchers increase visibility into the IPv6 address space. At the same time, our work further opens the door to explore security issues on IPv6. Researchers can begin to solve many security issues on IPv4.

XI. Acknowledgments

We would like to thank Zhizhu Liu, Xiaowen Quan, Oliver Gasser, and the anonymous reviewers for their insightful feedback and suggestions. This work is supported by the National Key Research and Development Program of China under Grant No.2017YFB0803004, 2018YFB1800200, 2018YFB1800205.

References

[1] R. Graham. (2018) MASSCAN. [Online]. Available: https://github.com/robertdavidgraham/masscan

[2] Z. Durumeric, E. Wustrow, and J. A. Halderman, "ZMap: Fast Internet-wide Scanning and Its Security Applications," in *Proceedings of the 22nd USENIX Security Symposium*, 2013, pp. 605–620.

[3] Z. Durumeric, M. Bailey, and J. A. Halderman, "An Internet-wide View of Internet-wide Scanning," in *Proceedings of the 23rd USENIX Security Symposium*, 2014, pp. 65–78.

[4] R. Beverly, "Yarrp'ing the Internet: Randomized High-Speed Active Topology Discovery," in *Proceedings of the 2016 Internet Measurement Conference*. ACM, 2016, pp. 413–420.

[5] Z. Durumeric, D. Adrian, A. Mirian, M. Bailey, and J. A. Halderman, "A Search Engine backed by Internet-wide Scanning," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2015, pp. 542–553.

[6] J. Czyz, M. J. Luckie, M. Allman, and M. Bailey, "Don't Forget to Lock the Back Door! A Characterization of IPv6 Network Security Policy," in *Proceedings of 23rd Annual Network and Distributed System Security Symposium*, 2016.

[7] P. Foremski, D. Plonka, and A. Berger, "Entropy/IP: Uncovering Structure in IPv6 Addresses," in *Proceedings of the 2016 Internet Measurement Conference*. ACM, 2016, pp. 167–181.

[8] O. Gasser, Q. Scheitle, P. Foremski, Q. Lone, M. Korczyński, S. D. Strowes, L. Hendriks, and G. Carle, "Clusters in the Expanse: Understanding and Unbiasing IPv6 Hitlists," in *Proceedings of the 2018 Internet Measurement Conference*. ACM, 2018, pp. 364–378.

[9] H. Dawood, "IPv6 Security Vulnerabilities," *International Journal of Information Security Science*, vol. 1, no. 4, pp. 100–105, 2012.

[10] L.Smith and I.Lipner. (2011) The IANA IPv4 Address Free Pool is Now Depleted. [Online]. Available: https://www.nro.net/ipv4-free-pool-depleted

[11] Google. (2019) IPv6 Adoption Statistics. [Online]. Available: https://www.google.com/intl/en/ipv6/statistics.html#tab=ipv6-adoption

[12] I. Launch. (2019) IPv6 is the New Normal. [Online]. Available: https://www.worldipv6launch.org/

[13] A. Murdock, F. Li, P. Bramsen, Z. Durumeric, and V. Paxson, "Target Generation for Internet-wide IPv6 Scanning," in *Proceedings of the 2017 Internet Measurement Conference*. ACM, 2017, pp. 242–253.

[14] Z. Liu, Y. Xiong, X. Liu, W. Xie, and P. Zhu, "6Tree: Efficient Dynamic Discovery of Active Addresses in the IPv6 Address Space," *Computer Networks*, vol. 155, pp. 31–46, 2019.

[15] F. Gont and T. Chown, "Network Reconnaissance in IPv6 Networks," RFC 7707, Mar. 2016.

[16] X. Li, M. Boucadair, C. Huitema, M. Bagnulo, and C. Bao, "IPv6 Addressing of IPv4/IPv6 Translators," RFC 6052, Oct. 2010.

[17] C. Huitema, "Teredo: Tunneling IPv6 over UDP through Network Address Translations (NATs)," RFC 4380, Feb. 2006.

[18] R. M. Hinden and S. E. Deering, "IP Version 6 Addressing Architecture," RFC 4291, Feb. 2006.

[19] S. Kawamura and M. Kawashima, "A Recommendation for IPv6 Address Text Representation," RFC 5952, Aug. 2010.

[20] S. D. Strowes, "Bootstrapping Active IPv6 Measurement with IPv4 and Public DNS," *arXiv preprint arXiv:1710.08536*, 2017.

[21] T. Fiebig, K. Borgolte, S. Hao, C. Kruegel, and G. Vigna, "Something from Nothing (There): Collecting Global IPv6 Datasets from DNS," in *International Conference on Passive and Active Network Measurement*. Springer, 2017, pp. 30–43.

[22] K. Borgolte, S. Hao, T. Fiebig, and G. Vigna, "Enumerating Active IPv6 Hosts for Large-scale Security Scans via DNSSEC-signed Reverse Zones," in *Proceedings of the 2018 IEEE Symposium on Security and Privacy*. IEEE, 2018, pp. 770–784.

[23] O. Gasser, Q. Scheitle, S. Gebhard, and G. Carle, "Scanning the IPv6 Internet: Towards a Comprehensive Hitlist," *arXiv preprint arXiv:1607.05179*, 2016.

[24] J. Ullrich, P. Kieseberg, K. Krombholz, and E. Weippl, "On Reconnaissance with IPv6: A Pattern-based Scanning Approach," in *2015 10th International Conference on Availability, Reliability and Security*. IEEE, 2015, pp. 186–192.

[25] J. Amann, O. Gasser, Q. Scheitle, L. Brent, G. Carle, and R. Holz, "Mission Accomplished?: HTTPS Security after Diginotar," in *Proceedings of the 2017 Internet Measurement Conference*. ACM, 2017, pp. 325–340.

[26] O. Gasser, B. Hof, M. Helm, M. Korczynski, R. Holz, and G. Carle, "In Log We Trust: Revealing Poor Security Practices with Certificate Transparency Logs and Internet Measurements," in *Proceedings of the International Conference on Passive and Active Network Measurement*. Springer, 2018, pp. 173–185.

[27] Q. Scheitle, T. Chung, J. Hiller, O. Gasser, J. Naab, R. van Rijswijk-Deij, O. Hohlfeld, R. Holz, D. Choffnes, A. Mislove *et al.*, "A First Look at Certification Authority Authorization (CAA)," *ACM SIGCOMM Computer Communication Review*, vol. 48, no. 2, pp. 10–23, 2018.

[28] R. P. Sonar. (2019) Forward DNS Data. [Online]. Available: https://opendata.rapid7.com/sonar.fdns_v2/

[29] M. Bryant. (2019) TLD AXFR Transfers. [Online]. Available: https://github.com/mandatoryprogrammer/TLDR

[30] A. Yeow. (2019) Bitnodes API. [Online]. Available: https://bitnodes.earn.com/

[31] RIPE NCC. (2019) IPMap. [Online]. Available: https://ftp.ripe.net/ripe/ipmap/

[32] J. Wu, "CNGI project and CERNET2," in *APCC/MDMC'04. The 2004 Joint Conference of the 10th Asia-Pacific Conference on Communications and the 5th International Symposium on Multi-Dimensional Mobile Communications Proceeding*, vol. 1. IEEE, 2004, pp. 0_13–0_14.

[33] M. Luckie, "Scamper: A Scalable and Extensible Packet Prober for Active Measurement of the Internet," in *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*. ACM, 2010, pp. 239–245.

[34] Cogent Communications, Inc. (2020) Cogent. [Online]. Available: https://www.cogentco.com

[35] C. Partridge and M. Allman, "Ethical Considerations in Network Measurement Papers," *Communications of the ACM*, vol. 59, no. 10, pp. 58–64, 2016.

[36] D. Dittrich, E. Kenneally *et al.*, "The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research," US Department of Homeland Security, Tech. Rep., 2012.